

Lab 01

Scenario

Consider the need to keep the main thread free to handle user interface and other application issues. Write a worker thread to sum the total of a large array of integer values.

Requirements

1. Write the solution using `std::thread` and manage the conclusion of the thread work correctly.
2. Re-write the solution using `std::jthread` and compare the complexity of the code with the first solution.

Lab 02

Scenario

Consider the need to keep the main thread free to handle user interface and other application issues. Divide up the summation work between two worker threads. Each can be given half of the large array to work on, and should update a shared total variable when done. Be sure to synchronize access to the total so that a lock is obtained before incrementing the value.

Requirements

1. Write the solution using `std::mutex` and manage the shared data during the thread work.
2. Re-write the solution using `std::lock_guard` to ensure the release of the resource no matter what for each thread.

Lab 03

Scenario

Real-Time Online Auction System

This application simulates an online auction platform, where users can place bids on items in real-time. Auctions have time limits, and bids need to be processed, displayed, and updated live for all users participating in each auction.

Auctions are fast-paced and highly interactive. It is import to handle concurrency as the system must handle multiple bids arriving at the same time.

Requirements

1. Use threads to manage multiple auctions running concurrently. Each auction can be handled by a separate thread to simulate real-world concurrent bidding.
2. Synchronize with `std::mutex` and `std::lock_guard` to protect the shared bid data for each auction to ensure consistency when multiple users place bids simultaneously.
3. Use condition variables to manage the auction end times, notifying threads when an auction is complete.

Lab 04

Scenario

Real-Time Online Auction System Add support for `std::future` and `std::promise` for handling asynchronous tasks such as updating the highest bid and sending bid confirmations.

Requirements

1. Modify the `Auction` class to use `std::promise` to set the result when a new highest bid is placed.
2. Use `std::future` to retrieve the bid updates and confirmations asynchronously.
3. See bid confirmations and notifications asynchronously for each bidder, providing a realistic simulation of bid placement and result retrieval.

Lab05

Scenario

Introduce a **thread pool** to improve the design of this auction application significantly by managing threads more efficiently. Instead of creating a new thread for each bidding or auction task, use a thread pool to reuse a fixed set of threads. This approach reduces the overhead of creating and destroying threads repeatedly, making the application more scalable and efficient, especially under high load with multiple concurrent auctions and bids.

The addition of a thread pool improves the efficiency and scalability of the auction application by:

- Reducing thread management overhead.
- Preventing the application from creating an excessive number of threads.
- Simplifying task submission, as tasks are now managed by the thread pool rather than individual threads.

Requirements

1. Improved Resource Management with a thread pool to control the maximum number of threads, which helps prevent system overload from creating too many threads.
2. Reduced Overhead by reusing threads in the pool reduces the cost of repeatedly creating and destroying threads, improving performance.
3. Make the application more scalable with thread pool to make it easier to handle a large number of auctions and bids, as it limits the number of active threads and queues tasks, allowing the system to respond well under heavy load.

Lab 06

Scenario

It has been decided to add in a reporting feature to show the current status of the open auctions. Add this using reader-writer locking logic so that reading the reports does not block the recoding of bidding action.

Requirements

1. Add reader-writer locking to this design using `std::shared_mutex`, `std::unique_lock`, and `std::shared_lock` to improve the efficiency of concurrent reads and writes, especially for reporting the current status of active auctions. By allowing multiple threads to read data simultaneously (as long as no thread is writing), we can efficiently support a reporting feature that retrieves the current state of active auctions without blocking bidding or auction-ending updates.

1. The starting logic for the problem without any threading could look like this:

```
#include <iostream>
#include <chrono>
#include <random>
#include <array>

int main(int argc, char const *argv[])
{
    std::array<int, 1000> numbers;

    // Seed the random number generator
    std::random_device rd;
    std::mt19937 gen(rd());
    std::uniform_int_distribution<> distrib(1, 100);

    // Fill the array with random numbers
    for (int& num : numbers) {
        num = distrib(gen);
    }

    // Measure the start time
    auto start = std::chrono::high_resolution_clock::now();

    // // Calculate the sum
    int sum = 0;
    for (int num : numbers) {
        sum += num;
    }

    // Measure the end time
    auto end = std::chrono::high_resolution_clock::now();

    // Calculate the duration
    auto duration = std::chrono::duration_cast<std::chrono::milliseconds>(end - start);

    std::cout << "Sum: " << sum << std::endl;
    std::cout << "Time taken: " << duration.count() << " milliseconds" << std::endl;

    return 0;
}
```

```
}
```

2. Refactoring the calculation directly out to a thread could look like this:

```
#include <iostream>
#include <chrono>
#include <random>
#include <array>
#include <thread>

int main(int argc, char const *argv[])
{
    std::array<int, 1000> numbers;

    // Seed the random number generator
    std::random_device rd;
    std::mt19937 gen(rd());
    std::uniform_int_distribution<> distrib(1, 100);

    // Fill the array with random numbers
    for (int& num : numbers) {
        num = distrib(gen);
    }

    // Measure the start time
    auto start = std::chrono::high_resolution_clock::now();

    // // Calculate the sum
    int sum = 0;
    std::thread worker_thread([&]() {
        for (int num : numbers) {
            sum += num;
        }
    });

    worker_thread.join();
    // Measure the end time
    auto end = std::chrono::high_resolution_clock::now();

    // Calculate the duration
    auto duration = std::chrono::duration_cast<std::chrono::milliseconds>(end - start);
```

```

std::cout << "Sum: " << sum << std::endl;
std::cout << "Time taken: " << duration.count() << " milliseconds" << std::endl;

return 0;
}

```

3. Reworking the solution to use `jthread` to start with could look like this:

```

#include <iostream>
#include <chrono>
#include <random>
#include <array>
#include <thread>

int main(int argc, char const *argv[])
{
    std::array<int, 1000> numbers;

    // Seed the random number generator
    std::random_device rd;
    std::mt19937 gen(rd());
    std::uniform_int_distribution<> distrib(1, 100);

    // Fill the array with random numbers
    for (int& num : numbers) {
        num = distrib(gen);
    }

    // Measure the start time
    auto start = std::chrono::high_resolution_clock::now();

    // // Calculate the sum
    int sum = 0;
    std::jthread worker_thread([&]() {
        for (int num : numbers) {
            sum += num;
        }
    });

    worker_thread.join();

    // Measure the end time

```

```

auto end = std::chrono::high_resolution_clock::now();

// Calculate the duration
auto duration = std::chrono::duration_cast<std::chrono::milliseconds>(end - start);

std::cout << "Sum: " << sum << std::endl;
std::cout << "Time taken: " << duration.count() << " milliseconds" << std::endl;

return 0;
}

```

4. This still calls join. If that call is removed the results will be incorrect.

Sum: 0

Time taken: 0 milliseconds

5. The problem is having the jthread go out of scope and automatically join before printing out the results. Surround the jthread in a block scope to force the destructor.

```

#include <iostream>
#include <chrono>
#include <random>
#include <array>
#include <thread>

int main(int argc, char const *argv[])
{
    std::array<int, 1000> numbers;

    // Seed the random number generator
    std::random_device rd;
    std::mt19937 gen(rd());
    std::uniform_int_distribution<> distrib(1, 100);

    // Fill the array with random numbers
    for (int &num : numbers)
    {
        num = distrib(gen);
    }

    // Measure the start time
    auto start = std::chrono::high_resolution_clock::now();

```

```
// // Calculate the sum
int sum = 0;
{
    std::jthread worker_thread([&]()
                                {
        for (int num : numbers) {
            sum += num;
        } });
}

// Measure the end time
auto end = std::chrono::high_resolution_clock::now();

// Calculate the duration
auto duration = std::chrono::duration_cast<std::chrono::milliseconds>(end - start);

std::cout << "Sum: " << sum << std::endl;
std::cout << "Time taken: " << duration.count() << " milliseconds" << std::endl;

return 0;
}
```

1. Directly using the mutex the solution might look like the following:

```
#include <iostream>
#include <chrono>
#include <random>
#include <array>
#include <thread>
#include <mutex>

int main(int argc, char const *argv[])
{
    std::array<int, 1000> numbers;

    // Seed the random number generator
    std::random_device rd;
    std::mt19937 gen(rd());
    std::uniform_int_distribution<> distrib(1, 100);

    // Fill the array with random numbers
    for (int &num : numbers)
    {
        num = distrib(gen);
    }

    // Measure the start time
    auto start = std::chrono::high_resolution_clock::now();

    // Divide the work in half
    int chunk_size = numbers.size() / 2;

    // Calculate the sum
    std::mutex sum_mutex;
    int sum = 0;

    // Create two threads
    {
        std::jthread thread1([&](int start, int end){
            for(int i = start; i < end; ++i)
            {
                sum_mutex.lock();
                sum += numbers[i];
            }
        });
    }
}
```

```

        sum_mutex.unlock();
    }
}, 0, chunk_size);
std::jthread thread2([&](int start, int end){
    for(int i = start; i < end; ++i)
    {
        sum_mutex.lock();
        sum += numbers[i];
        sum_mutex.unlock();
    }
}, chunk_size, numbers.size());
}

// Measure the end time
auto end = std::chrono::high_resolution_clock::now();

// Calculate the duration
auto duration = std::chrono::duration_cast<std::chrono::milliseconds>(end - start);

std::cout << "Sum: " << sum << std::endl;
std::cout << "Time taken: " << duration.count() << " milliseconds" << std::endl;

return 0;
}

```

2. Refactoring this to use the lock_guard.

```

#include <iostream>
#include <chrono>
#include <random>
#include <array>
#include <thread>
#include <mutex>

int main(int argc, char const *argv[])
{
    std::array<int, 1000> numbers;

    // Seed the random number generator
    std::random_device rd;
    std::mt19937 gen(rd());
    std::uniform_int_distribution<> distrib(1, 100);
}

```

```

// Fill the array with random numbers
for (int &num : numbers)
{
    num = distrib(gen);
}

// Measure the start time
auto start = std::chrono::high_resolution_clock::now();

// Divide the work in half
int chunk_size = numbers.size() / 2;

// Calculate the sum
std::mutex sum_mutex;
int sum = 0;

// Create two threads
{
    std::jthread thread1([&](int start, int end){
        for(int i = start; i < end; ++i)
        {
            std::lock_guard<std::mutex> thread1_guard(sum_mutex);
            sum += numbers[i];
        }
    }, 0, chunk_size);
    std::jthread thread2([&](int start, int end){
        for(int i = start; i < end; ++i)
        {
            std::lock_guard<std::mutex> thread1_guard(sum_mutex);
            sum += numbers[i];
        }
    }, chunk_size, numbers.size());
}

// Measure the end time
auto end = std::chrono::high_resolution_clock::now();

// Calculate the duration
auto duration = std::chrono::duration_cast<std::chrono::milliseconds>(end - start);

std::cout << "Sum: " << sum << std::endl;
std::cout << "Time taken: " << duration.count() << " milliseconds" << std::endl;

```

```
return 0;
```

```
}
```

To meet the requirements of a **Real-Time Online Auction System** with concurrency support, we'll use multiple threads to simulate concurrent auctions, with each auction handled by a separate thread. `std::mutex` and `std::lock_guard` will be used to protect shared bid data, ensuring that only one thread can modify the bid data at a time. We'll also use `std::condition_variable` to signal when an auction has ended, allowing other threads to proceed based on that event.

Here's how the solution might look:

```
#include <iostream>
#include <thread>
#include <mutex>
#include <condition_variable>
#include <map>
#include <string>
#include <chrono>
#include <vector>
#include <atomic>

class Auction {
public:
    Auction(std::string item, int duration_seconds)
        : item_name(std::move(item)), duration(duration_seconds), highest_bid(0),
        auction_ended(false) {}

    // Function to place a bid on the item
    void placeBid(int bid, const std::string& bidder) {
        std::lock_guard<std::mutex> lock(bid_mutex);
        if (auction_ended) {
            std::cout << "Auction for " << item_name << " has ended. No more bids allowed.\n";
            return;
        }
        if (bid > highest_bid) {
            highest_bid = bid;
            highest_bidder = bidder;
            std::cout << "New highest bid for " << item_name << ": " << bid << " by " << bidder <<
std::endl;
        } else {
            std::cout << "Bid of " << bid << " by " << bidder << " was too low for " << item_name <<
"\n";
        }
    }
};
```

```

}

// Function to run the auction with a time limit
void startAuction() {
    std::cout << "Starting auction for " << item_name << " lasting " << duration << "
seconds.\n";
    auto start_time = std::chrono::steady_clock::now();

    // Auction loop
    while (std::chrono::steady_clock::now() - start_time < std::chrono::seconds(duration)) {
        std::this_thread::sleep_for(std::chrono::milliseconds(100)); // Simulate time passing
    }

    // End the auction
    {
        std::lock_guard<std::mutex> lock(bid_mutex);
        auction_ended = true;
    }
    cv.notify_all(); // Notify all waiting threads that the auction has ended
    std::cout << "Auction for " << item_name << " has ended.\n";
}

// Function to get the final result of the auction
void waitForAuctionEnd() {
    std::unique_lock<std::mutex> lock(bid_mutex);
    cv.wait(lock, [this] { return auction_ended; });

    if (highest_bid > 0) {
        std::cout << "Auction result for " << item_name << ": Sold to " << highest_bidder
        << " for " << highest_bid << std::endl;
    } else {
        std::cout << "Auction result for " << item_name << ": No bids received.\n";
    }
}

private:
    std::string item_name;
    int duration;
    int highest_bid;
    std::string highest_bidder;
    bool auction_ended;
    std::mutex bid_mutex;
    std::condition_variable cv;
};

```

```

// Simulate bidding by different users
void simulateBidding(Auction& auction, const std::string& bidder, int max_bid) {
    for (int bid = 10; bid <= max_bid; bid += 10) {
        auction.placeBid(bid, bidder);
        std::this_thread::sleep_for(std::chrono::milliseconds(150)); // Simulate time between bids
    }
}

int main() {
    Auction auction1("Painting", 5); // 5-second auction
    Auction auction2("Antique Vase", 7); // 7-second auction

    // Start auctions in separate threads
    std::thread auction_thread1(&Auction::startAuction, &auction1);
    std::thread auction_thread2(&Auction::startAuction, &auction2);

    // Simulate bidding in separate threads for each auction
    std::thread bidder1(simulateBidding, std::ref(auction1), "Alice", 100);
    std::thread bidder2(simulateBidding, std::ref(auction1), "Bob", 80);
    std::thread bidder3(simulateBidding, std::ref(auction2), "Charlie", 90);
    std::thread bidder4(simulateBidding, std::ref(auction2), "Dave", 120);

    // Wait for auctions to end and display results
    std::thread result_thread1(&Auction::waitForAuctionEnd, &auction1);
    std::thread result_thread2(&Auction::waitForAuctionEnd, &auction2);

    // Join all threads
    auction_thread1.join();
    auction_thread2.join();
    bidder1.join();
    bidder2.join();
    bidder3.join();
    bidder4.join();
    result_thread1.join();
    result_thread2.join();

    return 0;
}

```

Explanation of the Code

1. Auction Class:

- `Auction` represents an individual auction with an item name, auction duration, highest bid, highest bidder, and a flag `auction_ended` to indicate if the auction has finished.
- The `bid_mutex` protects shared data (`highest_bid`, `highest_bidder`, and `auction_ended`), while the `cv` condition variable is used to notify when the auction ends.

2. `placeBid` Method:

- This method allows a user to place a bid on the item. It uses `std::lock_guard` to ensure thread-safe access to bid-related data.
- If the auction has ended (`auction_ended == true`), further bids are rejected. If the bid is higher than the current highest bid, it is recorded as the new highest bid.

3. `startAuction` Method:

- This method simulates running an auction with a time limit. It sleeps periodically to simulate the passage of time.
- Once the auction duration has passed, it sets `auction_ended` to `true` and calls `cv.notify_all()` to inform all threads waiting on `cv` that the auction is complete.

4. `waitForAuctionEnd` Method:

- This method waits for the auction to end using a condition variable. Once notified, it checks the highest bid and displays the final auction result.

5. `Main Function`:

- Two `Auction` objects are created, each with a different item and duration.
- Each auction is started in a separate thread (`auction_thread1`, `auction_thread2`).
- Multiple bidders are simulated with threads (`bidder1`, `bidder2`, etc.) that place bids at intervals.
- Once the auction has ended, results are fetched by `result_thread1` and `result_thread2`, which call `waitForAuctionEnd` to display the auction outcome.

Key Points

- **Concurrency:**
 - Each auction and each bidder is handled in separate threads, simulating concurrent bidding across multiple auctions.
- **Mutex and Condition Variable Usage:**
 - A `std::mutex` (`bid_mutex`) protects bid data within the auction, and `std::lock_guard` ensures thread-safe updates.
 - A `std::condition_variable` (`cv`) is used to notify when an auction ends, allowing any threads waiting for the end of the auction to proceed.
- **Result Synchronization:**

- After the auction time ends, all bidders stop bidding, and results are displayed in separate result threads.

This code demonstrates how to handle concurrent bidding in a simulated online auction system using C++ concurrency primitives, ensuring that bid data is safely updated and that auction end notifications are handled appropriately.

Here's an updated version of the auction code that includes `std::promise` and `std::future`:

```
#include <iostream>
#include <thread>
#include <mutex>
#include <condition_variable>
#include <map>
#include <string>
#include <chrono>
#include <vector>
#include <atomic>
#include <future>

class Auction {
public:
    Auction(std::string item, int duration_seconds)
        : item_name(std::move(item)), duration(duration_seconds), highest_bid(0),
        auction_ended(false) {}

    // Function to place a bid on the item
    std::future<std::string> placeBid(int bid, const std::string& bidder) {
        std::lock_guard<std::mutex> lock(bid_mutex);

        std::promise<std::string> bid_promise;
        std::future<std::string> bid_future = bid_promise.get_future();

        if (auction_ended) {
            bid_promise.set_value("Auction for " + item_name + " has ended. No more bids allowed.");
            return bid_future;
        }

        if (bid > highest_bid) {
            highest_bid = bid;
            highest_bidder = bidder;
            std::string confirmation = "New highest bid for " + item_name + ": " +
std::to_string(bid) + " by " + bidder;
            bid_promise.set_value(confirmation); // Set confirmation message in promise
            std::cout << confirmation << std::endl;
        } else {
            std::string message = "Bid of " + std::to_string(bid) + " by " + bidder + " was too low
for " + item_name;
```

```

        bid_promise.set_value(message); // Set failure message in promise
        std::cout << message << std::endl;
    }

    return bid_future; // Return the future to the caller
}

// Function to run the auction with a time limit
void startAuction() {
    std::cout << "Starting auction for " << item_name << " lasting " << duration << "
seconds.\n";
    auto start_time = std::chrono::steady_clock::now();

    // Auction loop
    while (std::chrono::steady_clock::now() - start_time < std::chrono::seconds(duration)) {
        std::this_thread::sleep_for(std::chrono::milliseconds(100)); // Simulate time passing
    }

    // End the auction
    {
        std::lock_guard<std::mutex> lock(bid_mutex);
        auction_ended = true;
    }
    cv.notify_all(); // Notify all waiting threads that the auction has ended
    std::cout << "Auction for " << item_name << " has ended.\n";
}

// Function to get the final result of the auction
void waitForAuctionEnd() {
    std::unique_lock<std::mutex> lock(bid_mutex);
    cv.wait(lock, [this] { return auction_ended; });

    if (highest_bid > 0) {
        std::cout << "Auction result for " << item_name << ": Sold to " << highest_bidder
        << " for " << highest_bid << std::endl;
    } else {
        std::cout << "Auction result for " << item_name << ": No bids received.\n";
    }
}

private:
    std::string item_name;
    int duration;
    int highest_bid;

```

```

    std::string highest_bidder;
    bool auction_ended;
    std::mutex bid_mutex;
    std::condition_variable cv;
};

// Simulate bidding by different users
void simulateBidding(Auction& auction, const std::string& bidder, int max_bid) {
    for (int bid = 10; bid <= max_bid; bid += 10) {
        std::future<std::string> result = auction.placeBid(bid, bidder);

        // Asynchronously get the bid confirmation
        std::cout << "Confirmation for " << bidder << ": " << result.get() << std::endl;

        std::this_thread::sleep_for(std::chrono::milliseconds(150)); // Simulate time between bids
    }
}

int main() {
    Auction auction1("Painting", 5); // 5-second auction
    Auction auction2("Antique Vase", 7); // 7-second auction

    // Start auctions in separate threads
    std::thread auction_thread1(&Auction::startAuction, &auction1);
    std::thread auction_thread2(&Auction::startAuction, &auction2);

    // Simulate bidding in separate threads for each auction
    std::thread bidder1(simulateBidding, std::ref(auction1), "Alice", 100);
    std::thread bidder2(simulateBidding, std::ref(auction1), "Bob", 80);
    std::thread bidder3(simulateBidding, std::ref(auction2), "Charlie", 90);
    std::thread bidder4(simulateBidding, std::ref(auction2), "Dave", 120);

    // Wait for auctions to end and display results
    std::thread result_thread1(&Auction::waitForAuctionEnd, &auction1);
    std::thread result_thread2(&Auction::waitForAuctionEnd, &auction2);

    // Join all threads
    auction_thread1.join();
    auction_thread2.join();
    bidder1.join();
    bidder2.join();
    bidder3.join();
    bidder4.join();
    result_thread1.join();
}

```

```
    result_thread2.join();

    return 0;
}
```

Explanation of Updates

1. Using `std::promise` and `std::future` in `placeBid`:

- The `placeBid` method now creates a `std::promise` for each bid attempt. This promise holds a confirmation message (or a rejection message) for each bid.
- The future associated with the promise (`bid_future`) is returned to the caller, allowing asynchronous retrieval of the bid result.

2. Setting the Promise Value:

- If the bid is higher than the current highest bid, the promise's value is set to a confirmation message, which can be retrieved asynchronously.
- If the bid is too low or if the auction has ended, the promise's value is set to a failure message, allowing the caller to receive immediate feedback on the bid's status.

3. Asynchronous Bid Confirmation:

- In the `simulateBidding` function, each call to `placeBid` returns a `std::future`. By calling `result.get()`, we asynchronously retrieve the bid confirmation message, which may indicate success or failure based on the bid value and auction state.
- Using `result.get()` in this way provides a non-blocking way for each bidder to be notified of their bid status.

4. Thread Safety and Synchronization:

- `std::mutex` and `std::lock_guard` ensure thread-safe access to the highest bid and highest bidder data.
- `std::condition_variable` is used to signal when the auction ends, allowing the main thread to display final results.

Summary

By incorporating `std::promise` and `std::future`, this version of the auction system allows for asynchronous bid updates and confirmations, enhancing the responsiveness and interactivity of the system. Each bid can be confirmed in real-time, providing immediate feedback to bidders, while ensuring that the highest bid data remains protected with `std::mutex` and `std::lock_guard`. The use of `std::condition_variable` continues to handle auction end notifications, keeping the system synchronized and efficient.

Revised Design with a Thread Pool

In this revised design:

- **Thread Pool:** We'll introduce a simple thread pool to handle auction tasks and bid processing tasks.
- **Task Submission:** Auction start, bidding, and result processing tasks will be submitted to the thread pool instead of spawning individual threads.
- **Synchronization:** We'll continue using `std::mutex`, `std::lock_guard`, and `std::condition_variable` to protect shared data and notify threads about auction end events.

Here's a basic thread pool implementation and how it would be integrated into our auction application.

Step 1: Implement a Simple Thread Pool

```
#include <vector>
#include <queue>
#include <thread>
#include <future>
#include <functional>
#include <condition_variable>

class ThreadPool {
public:
    ThreadPool(size_t num_threads) {
        for (size_t i = 0; i < num_threads; ++i) {
            workers.emplace_back([this] {
                while (true) {
                    std::function<void()> task;
                    {
                        std::unique_lock<std::mutex> lock(queue_mutex);
                        condition.wait(lock, [this] { return stop || !tasks.empty(); });
                        if (stop && tasks.empty()) return;
                        task = std::move(tasks.front());
                        tasks.pop();
                    }
                    task(); // Execute the task
                }
            });
        }
    }
};
```

```

}

template<class F, class... Args>
auto enqueue(F&& f, Args&&... args) -> std::future<typename std::result_of<F(Args...)>::type> {
    using return_type = typename std::result_of<F(Args...)>::type;
    auto task = std::make_shared<std::packaged_task<return_type()>>(std::bind(std::forward<F>
(f), std::forward<Args>(args)...));
    std::future<return_type> result = task->get_future();
    {
        std::unique_lock<std::mutex> lock(queue_mutex);
        if (stop) throw std::runtime_error("enqueue on stopped ThreadPool");
        tasks.emplace([task]() { (*task)(); });
    }
    condition.notify_one();
    return result;
}

~ThreadPool() {
    {
        std::unique_lock<std::mutex> lock(queue_mutex);
        stop = true;
    }
    condition.notify_all();
    for (std::thread &worker : workers) worker.join();
}

private:
    std::vector<std::thread> workers;
    std::queue<std::function<void()>> tasks;
    std::mutex queue_mutex;
    std::condition_variable condition;
    bool stop = false;
};

```

Step 2: Modify the Auction Application to Use the Thread Pool

Now we'll refactor the auction application to submit auction and bid tasks to the thread pool.

```

#include <iostream>
#include <thread>
#include <mutex>
#include <condition_variable>
#include <string>

```

```

#include <chrono>
#include <future>
#include <vector>

class Auction {
public:
    Auction(std::string item, int duration_seconds)
        : item_name(std::move(item)), duration(duration_seconds), highest_bid(0),
    auction_ended(false) {}

    // Function to place a bid on the item
    std::future<std::string> placeBid(int bid, const std::string& bidder, ThreadPool& thread_pool) {
        return thread_pool.enqueue([this, bid, bidder]() {
            std::lock_guard<std::mutex> lock(bid_mutex);
            if (auction_ended) {
                return "Auction for " + item_name + " has ended. No more bids allowed.";
            }
            if (bid > highest_bid) {
                highest_bid = bid;
                highest_bidder = bidder;
                std::string confirmation = "New highest bid for " + item_name + ": " +
std::to_string(bid) + " by " + bidder;
                std::cout << confirmation << std::endl;
                return confirmation;
            } else {
                std::string message = "Bid of " + std::to_string(bid) + " by " + bidder + " was too
low for " + item_name;
                std::cout << message << std::endl;
                return message;
            }
        });
    }

    // Function to run the auction with a time limit
    void startAuction(ThreadPool& thread_pool) {
        thread_pool.enqueue([this] {
            std::cout << "Starting auction for " << item_name << " lasting " << duration << "
seconds.\n";
            auto start_time = std::chrono::steady_clock::now();

            while (std::chrono::steady_clock::now() - start_time < std::chrono::seconds(duration)) {
                std::this_thread::sleep_for(std::chrono::milliseconds(100));
            }
        });
    }
};

```

```

        {
            std::lock_guard<std::mutex> lock(bid_mutex);
            auction_ended = true;
        }
        cv.notify_all();
        std::cout << "Auction for " << item_name << " has ended.\n";
    });
}

// Function to get the final result of the auction
void waitForAuctionEnd(ThreadPool& thread_pool) {
    thread_pool.enqueue([this] {
        std::unique_lock<std::mutex> lock(bid_mutex);
        cv.wait(lock, [this] { return auction_ended; });
        if (highest_bid > 0) {
            std::cout << "Auction result for " << item_name << ": Sold to " << highest_bidder
                << " for " << highest_bid << std::endl;
        } else {
            std::cout << "Auction result for " << item_name << ": No bids received.\n";
        }
    });
}

private:
    std::string item_name;
    int duration;
    int highest_bid;
    std::string highest_bidder;
    bool auction_ended;
    std::mutex bid_mutex;
    std::condition_variable cv;
};

// Simulate bidding by different users
void simulateBidding(Auction& auction, const std::string& bidder, int max_bid, ThreadPool&
thread_pool) {
    for (int bid = 10; bid <= max_bid; bid += 10) {
        std::future<std::string> result = auction.placeBid(bid, bidder, thread_pool);
        std::cout << "Confirmation for " << bidder << ": " << result.get() << std::endl;
        std::this_thread::sleep_for(std::chrono::milliseconds(150));
    }
}

int main() {

```

```

ThreadPool thread_pool(4); // Create a thread pool with 4 threads

Auction auction1("Painting", 5);
Auction auction2("Antique Vase", 7);

// Start auctions using the thread pool
auction1.startAuction(thread_pool);
auction2.startAuction(thread_pool);

// Simulate bidding using the thread pool
std::thread bidder1(simulateBidding, std::ref(auction1), "Alice", 100, std::ref(thread_pool));
std::thread bidder2(simulateBidding, std::ref(auction1), "Bob", 80, std::ref(thread_pool));
std::thread bidder3(simulateBidding, std::ref(auction2), "Charlie", 90, std::ref(thread_pool));
std::thread bidder4(simulateBidding, std::ref(auction2), "Dave", 120, std::ref(thread_pool));

// Wait for auctions to end and display results
auction1.waitForAuctionEnd(thread_pool);
auction2.waitForAuctionEnd(thread_pool);

// Join all threads
bidder1.join();
bidder2.join();
bidder3.join();
bidder4.join();

return 0;
}

```

Explanation of the Updated Code

1. Thread Pool:

- We've added a `ThreadPool` class that manages a fixed number of worker threads. It allows tasks to be enqueued and executed by any available thread in the pool.

2. Task Submission:

- The `startAuction`, `placeBid`, and `waitForAuctionEnd` functions now accept a `ThreadPool` reference and submit their work to the thread pool using `enqueue`.
- This keeps the number of threads manageable and allows the application to scale gracefully as we add more auctions or bidders.

3. Reduced Overhead:

- By using the thread pool, we avoid creating and destroying threads repeatedly for each bid or auction, reducing overhead and improving performance, especially with multiple auctions and bidders.

How Reader-Writer Locking Improves the Design

1. **Concurrent Reads:** With `std::shared_mutex`, multiple threads can hold a `std::shared_lock` simultaneously, allowing concurrent read access. This is ideal for scenarios where we frequently need to read the auction status but less frequently update it.
2. **Exclusive Writes:** Only one thread at a time can hold a `std::unique_lock` on the `std::shared_mutex`, ensuring exclusive write access. This is necessary for operations like placing a bid or ending the auction, which modify the auction state.
3. **Efficient Reporting:** Reader-writer locking allows the system to provide real-time reporting on the current state of active auctions without interfering with ongoing bidding activities.

Updated Design with `std::shared_mutex` for Reporting

Below is the updated code with `std::shared_mutex` added to allow concurrent reads. We'll also add a `reportCurrentState` function to display the status of active auctions.

```
#include <iostream>
#include <thread>
#include <mutex>
#include <shared_mutex>
#include <condition_variable>
#include <string>
#include <chrono>
#include <future>
#include <vector>
#include <map>

class Auction {
public:
    Auction(std::string item, int duration_seconds)
        : item_name(std::move(item)), duration(duration_seconds), highest_bid(0),
        auction_ended(false) {}

    // Function to place a bid on the item
    std::future<std::string> placeBid(int bid, const std::string& bidder, ThreadPool& thread_pool) {
        return thread_pool.enqueue([this, bid, bidder]() {
            std::unique_lock<std::shared_mutex> lock(bid_mutex); // Exclusive lock for modifying bid
            data

            if (auction_ended) {
                return "Auction for " + item_name + " has ended. No more bids allowed.";
            }
            if (bid > highest_bid) {
```

```

        highest_bid = bid;
        highest_bidder = bidder;
        std::string confirmation = "New highest bid for " + item_name + ": " +
std::to_string(bid) + " by " + bidder;
        std::cout << confirmation << std::endl;
        return confirmation;
    } else {
        std::string message = "Bid of " + std::to_string(bid) + " by " + bidder + " was too
low for " + item_name;
        std::cout << message << std::endl;
        return message;
    }
});
}

// Function to run the auction with a time limit
void startAuction(ThreadPool& thread_pool) {
    thread_pool.enqueue([this] {
        std::cout << "Starting auction for " << item_name << " lasting " << duration << "
seconds.\n";
        auto start_time = std::chrono::steady_clock::now();

        while (std::chrono::steady_clock::now() - start_time < std::chrono::seconds(duration)) {
            std::this_thread::sleep_for(std::chrono::milliseconds(100));
        }

        {
            std::unique_lock<std::shared_mutex> lock(bid_mutex); // Exclusive lock to end
auction
            auction_ended = true;
        }
        cv.notify_all();
        std::cout << "Auction for " << item_name << " has ended.\n";
    });
}

// Function to get the final result of the auction
void waitForAuctionEnd(ThreadPool& thread_pool) {
    thread_pool.enqueue([this] {
        std::unique_lock<std::shared_mutex> lock(bid_mutex);
        cv.wait(lock, [this] { return auction_ended; });
        if (highest_bid > 0) {
            std::cout << "Auction result for " << item_name << ": Sold to " << highest_bidder
<< " for " << highest_bid << std::endl;

```

```

        } else {
            std::cout << "Auction result for " << item_name << ": No bids received.\n";
        }
    });
}

// Function to report the current state of the auction (for active reporting)
std::string reportStatus() const {
    std::shared_lock<std::shared_mutex> lock(bid_mutex); // Shared lock for reading data
    if (auction_ended) {
        return "Auction for " + item_name + " has ended.";
    } else {
        return "Current highest bid for " + item_name + " is " + std::to_string(highest_bid)
            + " by " + highest_bidder;
    }
}

private:
    std::string item_name;
    int duration;
    int highest_bid;
    std::string highest_bidder;
    bool auction_ended;
    mutable std::shared_mutex bid_mutex; // Shared mutex for reader-writer locking
    std::condition_variable_any cv; // Use condition_variable_any for shared_mutex support
};

// Function to report the state of all active auctions
void reportAllAuctions(const std::vector<std::shared_ptr<Auction>>& auctions) {
    for (const auto& auction : auctions) {
        std::cout << auction->reportStatus() << std::endl;
    }
}

// Simulate bidding by different users
void simulateBidding(Auction& auction, const std::string& bidder, int max_bid, ThreadPool&
thread_pool) {
    for (int bid = 10; bid <= max_bid; bid += 10) {
        std::future<std::string> result = auction.placeBid(bid, bidder, thread_pool);
        std::cout << "Confirmation for " << bidder << ": " << result.get() << std::endl;
        std::this_thread::sleep_for(std::chrono::milliseconds(150));
    }
}

```

```

int main() {
    ThreadPool thread_pool(4); // Create a thread pool with 4 threads

    auto auction1 = std::make_shared<Auction>("Painting", 5);
    auto auction2 = std::make_shared<Auction>("Antique Vase", 7);
    std::vector<std::shared_ptr<Auction>> auctions = {auction1, auction2};

    // Start auctions using the thread pool
    auction1->startAuction(thread_pool);
    auction2->startAuction(thread_pool);

    // Simulate bidding using the thread pool
    std::thread bidder1(simulateBidding, std::ref(*auction1), "Alice", 100, std::ref(thread_pool));
    std::thread bidder2(simulateBidding, std::ref(*auction1), "Bob", 80, std::ref(thread_pool));
    std::thread bidder3(simulateBidding, std::ref(*auction2), "Charlie", 90, std::ref(thread_pool));
    std::thread bidder4(simulateBidding, std::ref(*auction2), "Dave", 120, std::ref(thread_pool));

    // Periodic reporting of all active auctions
    std::thread reporter([&auctions] {
        while (true) {
            std::this_thread::sleep_for(std::chrono::seconds(1)); // Report every second
            reportAllAuctions(auctions);
        }
    });

    // Wait for auctions to end and display results
    auction1->waitForAuctionEnd(thread_pool);
    auction2->waitForAuctionEnd(thread_pool);

    // Join all threads
    bidder1.join();
    bidder2.join();
    bidder3.join();
    bidder4.join();
    reporter.detach(); // Detach reporter as it's a continuous task

    return 0;
}

```

Explanation of Changes

1. Reader-Writer Locking with `std::shared_mutex`:

- We've replaced the `std::mutex` in the `Auction` class with `std::shared_mutex` (assigned to `bid_mutex`) to allow multiple threads to read the auction state concurrently without blocking each other.

2. `std::shared_lock` for Read-Only Access:

- The `reportStatus` function acquires a `std::shared_lock` on `bid_mutex`, allowing it to read the auction data without blocking other readers. This allows the reporting thread to call `reportStatus` concurrently with other readers.

3. `std::unique_lock` for Exclusive Write Access:

- `placeBid`, `startAuction`, and `waitForAuctionEnd` still use `std::unique_lock` for exclusive write access, ensuring that these functions can modify the auction state safely.

4. New `reportAllAuctions` Function:

- This function iterates over a list of active auctions, calling `reportStatus` on each one to display the current highest bid and bidder without blocking bidding threads.
- We launch this function in a separate `reporter` thread that runs continuously to update the auction status every second.

Summary

Adding `std::shared_mutex`, `std::shared_lock`, and `std::unique_lock` allows us to implement efficient read access, which is essential for reporting on active auctions without interfering with ongoing bidding. This design allows multiple threads to read auction data concurrently while ensuring exclusive access for modifications, making the application more scalable and responsive for real-time updates and reporting.